

Questions:

Part01 Questions & Answers

1.Question: Why is it recommended to explicitly assign values to enum members in some cases?

Answer: Explicitly assigning values prevents breaking database records, external API contracts, or serialized data when new enum members are added or reordered in the future.

2. Question: What happens if you assign a value to an enum member that exceeds the underlying type's range?

Answer: The C# compiler throws a compilation error because a value outside the underlying type's storage bounds cannot be assigned without an explicit cast or causing an overflow error.

3. Question: What is the purpose of the virtual keyword when used with properties?

Answer: The virtual keyword allows derived classes to override the property implementation to customize getter or setter logic.

3. Question: Why can't you override a sealed property or method?

Answer: A sealed modifier explicitly stops the inheritance chain, preventing derived classes from further overriding the member to guarantee its implementation remains fixed.

4. Question: What is the key difference between static and object (instance) members?

Answer: Static members belong to the class itself and are shared across all instances without instantiation, whereas object (instance) members belong to specific object instances.

5. Question: Can you overload all operators in C#? Explain why or why not.

Answer: No, operators like &&, ||, =, ., ?:, new, sizeof, and typeof cannot be overloaded to preserve core C# logic, safety, and memory management guarantees.

6.Question: When should you consider changing the underlying type of an enum?

Answer: You should change it (e.g., to byte) when optimizing memory usage in large arrays, database persistence, or low-level network data transmission.

7.Question: Why can't a static class have instance constructors?

Answer: Static classes cannot be instantiated with new, so instance constructors are invalid. They can only contain parameterless static constructors managed automatically by the CLR.

8. Question: What is the difference between overriding Equals and == for object comparison in C# struct and class?

Answer: Overriding Equals defines logical value-based comparison, while == performs reference comparison by default for classes unless explicitly overloaded.

9. Question: Why is overriding ToString() beneficial when working with custom classes?

Answer: Overriding ToString() provides a readable textual representation of an object's internal state, simplifying debugging and logging.

10. Question: Can generics be constrained to specific types in C#? Provide an example.

Answer: Yes, generics use the where clause (e.g., where T : IComparable<T>) to restrict type parameters to specific base classes or interfaces.

11. Question: What are the key differences between generic methods and generic classes?

Answer: Generic methods define type parameters at the method level with automatic type inference, while generic classes define type parameters at the class level affecting all non-static members.

12. Question: Why might using a generic swap method be preferable to implementing custom methods for each type?

Answer: A generic swap method eliminates redundant code and guarantees compile-time type safety across all data types without creating multiple overloads.

13. Question: How can overriding Equals for the Department class improve the accuracy of searches?

Answer: Overriding Equals allows methods like SearchArray to match objects based on property value equality rather than memory reference equality.

14. Question: Why is == not implemented by default for structs?

Answer: The == operator is omitted by default in structs to prevent performance hits caused by field-by-field reflection operations on value types.

Part02 Conceptual Questions & Answers

1. Question: What do we mean by Generalization concept using Generics?

Answer: Generalization allows writing type-agnostic code using type placeholders (<T>), achieving code reusability, compile-time safety, and avoiding performance hits like boxing.

2. Question: What do we mean by hierarchy design in real business?

Answer: Hierarchy design structures business domain entities through inheritance and composition (e.g., BaseEntity -> User -> Employee), promoting DRY principles and centralized logic.

Part03 Bonus Questions & Answers

1. Question: What is covered in the Self Study Report?

Answer: A synthesis covering advanced OOP concepts, generic structures, stack vs heap memory management, type safety, and custom object equality in .NET.

2.Question: What is Event-Driven Programming?

Answer: A paradigm where execution flow is driven by user actions or system signals, implemented in C# using delegates and event keywords to decouple components.

3. Question: Why must operator overloading methods be declared as public static?

Answer:

1- Why static? Operators belong to the type itself rather than an individual instance. Expressions like `c1 + c2` invoke the method directly on the class (`ComplexNumber.operator+(c1, c2)`).

2- Why public? Operators must be accessible globally wherever expressions with these objects are evaluated outside the class.

3- Language Rule: C# specification enforces public static and requires at least one parameter to be of the enclosing type.